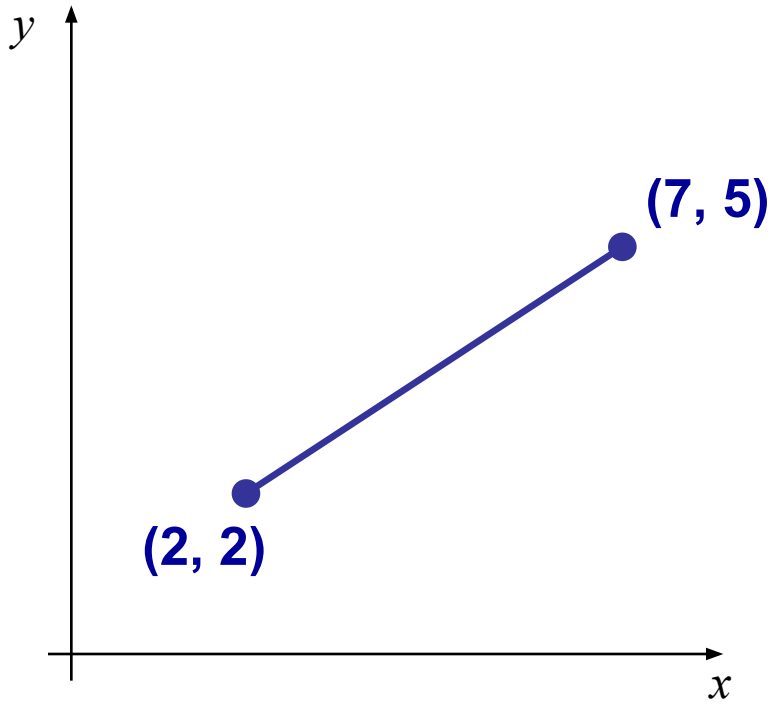
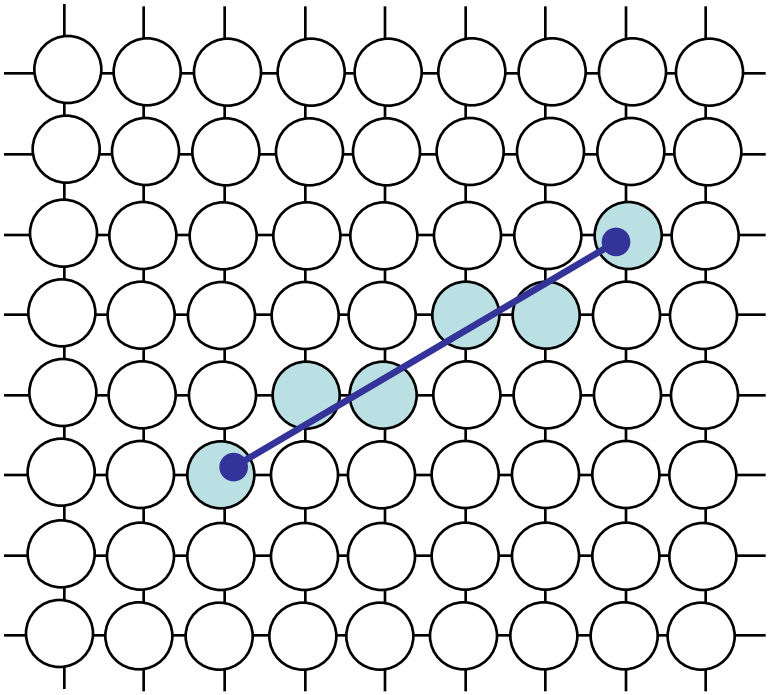
# The Problem Of Scan Conversion

A line segment in a scene is defined by the coordinate positions of the line end-points



# The Problem (cont…)



How do we choose which pixels to turn on?

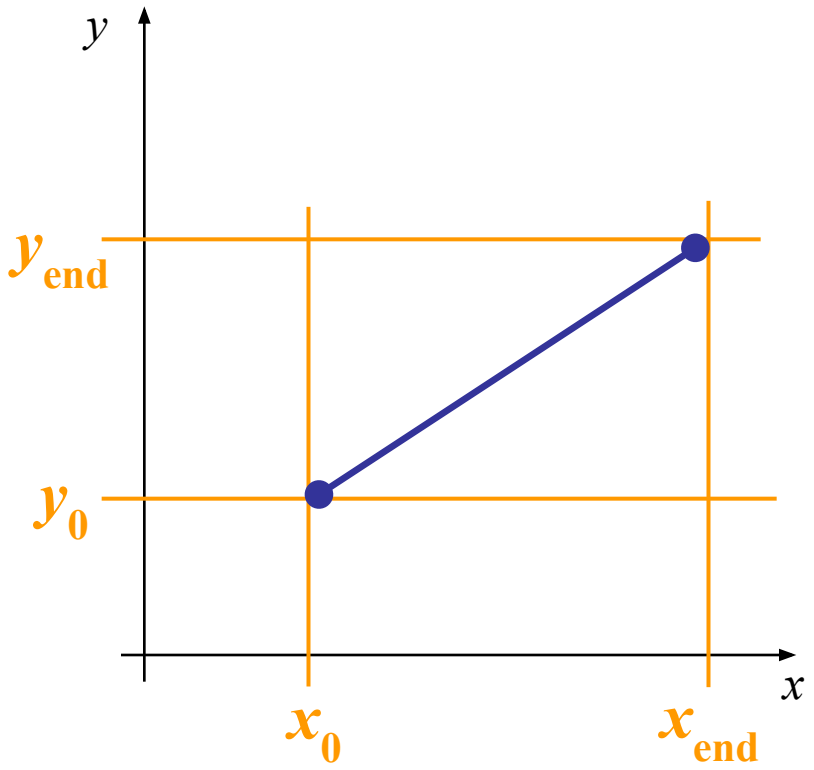
# Considerations

Considerations to keep in mind:

- The line has to look good
  - Avoid *jaggies*
- It has to be lightening fast!
  - How many lines need to be drawn in a typical scene?
  - This is going to come back to bite us again and again

# Line Equations

Let's quickly review the equations involved in drawing lines




Slope-intercept line equation:

$$y = m \cdot x + b$$

where:

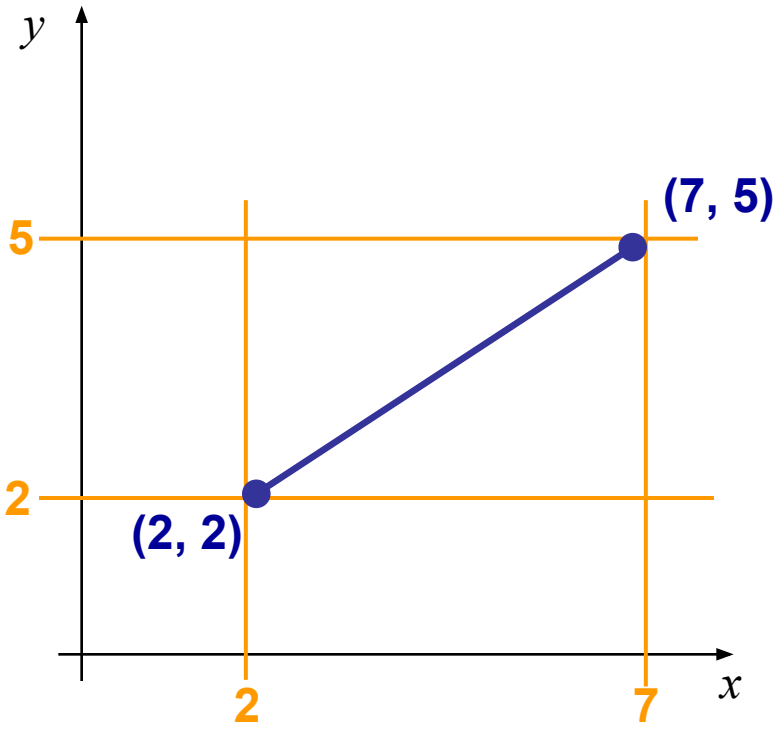
$$m = \frac{y_{end} - y_0}{x_{end} - x_0}$$

$$b = y_0 - m \cdot x_0$$

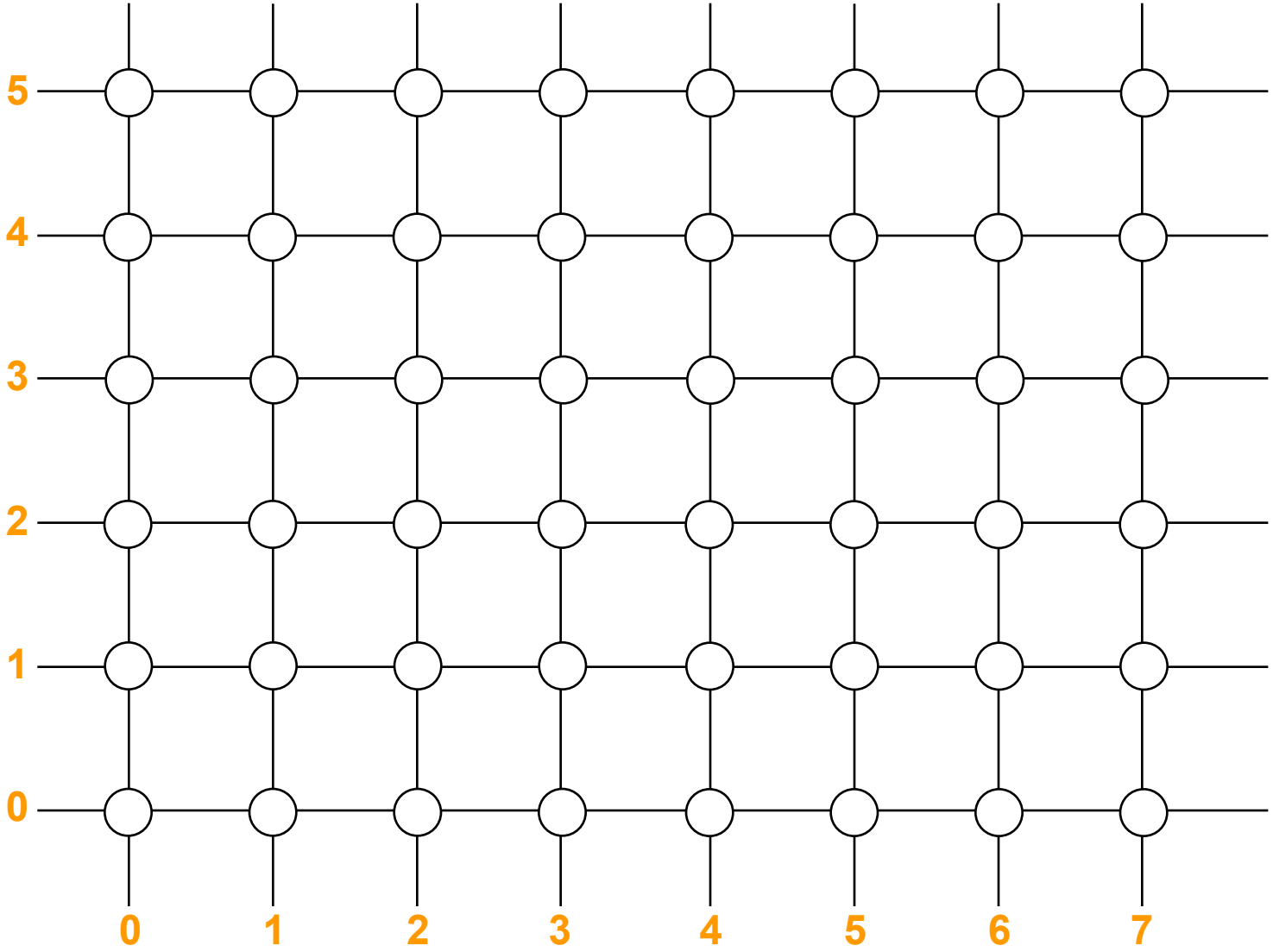
# A Very Simple Solution

We could simply work out the corresponding $y$ coordinate for each unit $x$ coordinate

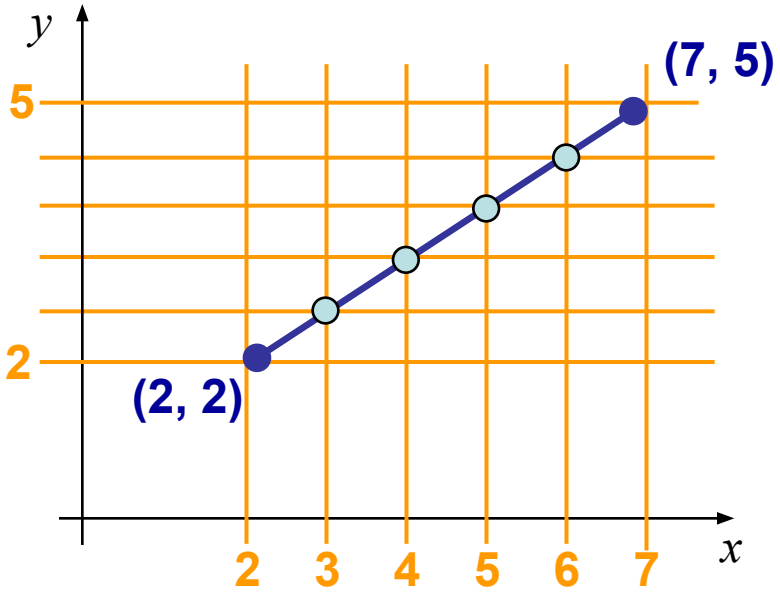
Let's consider the following example:



# A Very Simple Solution (cont…)



# A Very Simple Solution (cont…)




First work out $m$ and $b$:

$$m = \frac{5-2}{7-2} = \frac{3}{5}$$

$$b = 2 - \frac{3}{5} * 2 = \frac{4}{5}$$

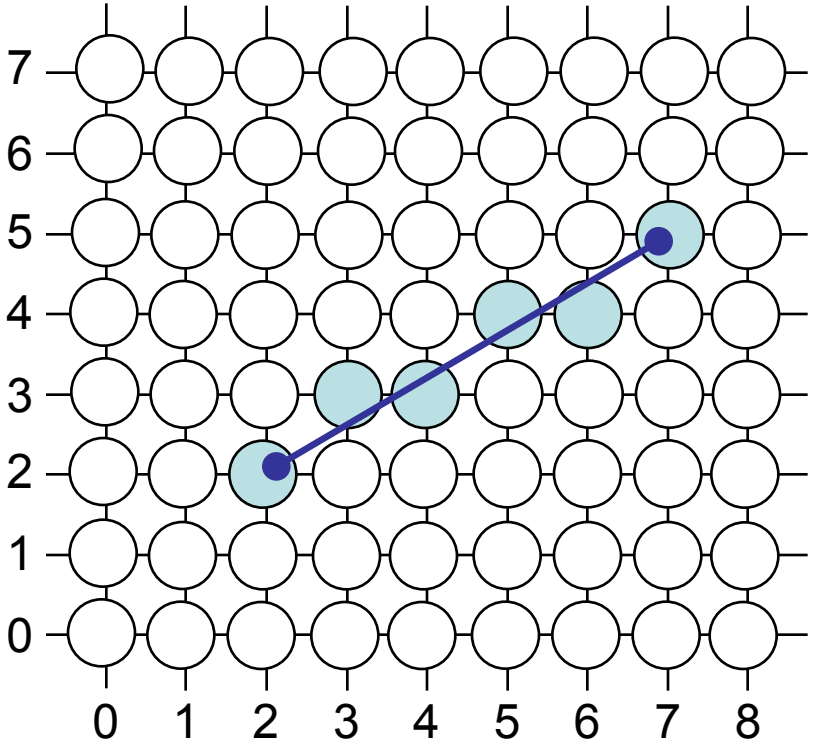
Now for each $x$ value work out the $y$ value:

$$y(3) = \frac{3}{5} \cdot 3 + \frac{4}{5} = 2\frac{3}{5} \qquad y(4) = \frac{3}{5} \cdot 4 + \frac{4}{5} = 3\frac{1}{5}$$

$$y(5) = \frac{3}{5} \cdot 5 + \frac{4}{5} = 3\frac{4}{5} \qquad y(6) = \frac{3}{5} \cdot 6 + \frac{4}{5} = 4\frac{2}{5}$$

# A Very Simple Solution (cont…)

Now just round off the results and turn on these pixels to draw our line




$$y(3) = 2\frac{3}{5} \approx 3$$

$$y(4) = 3\frac{1}{5} \approx 3$$

$$y(5) = 3\frac{4}{5} \approx 4$$

$$y(6) = 4\frac{2}{5} \approx 4$$

# A Very Simple Solution (cont…)

However, this approach is just way too slow

In particular look out for:

- The equation $y = mx + b$ requires the multiplication of $m$ by $x$
- Rounding off the resulting $y$ coordinates

We need a faster solution

# A Quick Note About Slopes

In the previous example we chose to solve the parametric line equation to give us the $y$ coordinate for each unit $x$ coordinate

What if we had done it the other way around?

So this gives us: $$x = \frac{y - b}{m}$$

where: $$m = \frac{y_{end} - y_0}{x_{end} - x_0} \quad \text{and} \quad b = y_0 - m \cdot x_0$$

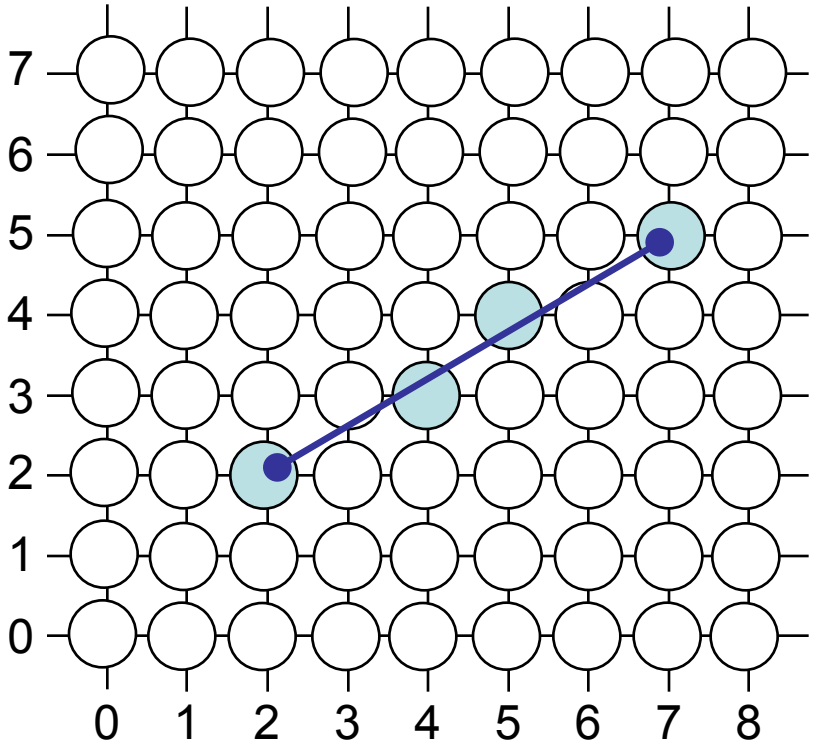
# A Quick Note About Slopes (cont…)

Leaving out the details this gives us:

$$x(3) = 3\frac{2}{3} \approx 4 \qquad x(4) = 5\frac{1}{3} \approx 5$$

We can see easily that this line doesn't look very good!

We choose which way to work out the line pixels based on the slope of the line



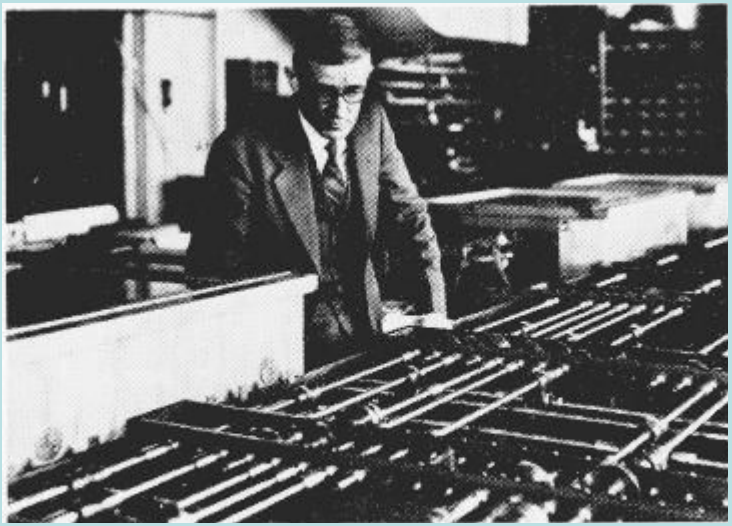
# A Quick Note About Slopes

- If **|m|<1**, then for every integer value of x between and excluding x1 and x2, calculate the corresponding value of y using equation **Δy = m Δx** & scan convert (x,y).
- If **|m|>1**, then for every integer value of y between and excluding y1 and y2, calculate the corresponding value of x using equation **Δx = Δy/m** & scan convert (x,y).
- If **|m|=1**, **Δx = Δy.** In each case, a smooth line with slope m is generated between the specific endpoints.

# The DDA Algorithm

The *digital differential analyzer* (DDA) algorithm takes an incremental approach in order to speed up scan conversion

Simply calculate $y_{k+1}$ based on $y_k$



The original differential analyzer was a physical machine developed by Vannevar Bush at MIT in the 1930's in order to solve ordinary differential equations.

More information here.

# The DDA Algorithm (cont…)

Consider the list of points that we determined for the line in our previous example:

$(2, 2), (3, 2^3/_5), (4, 3^1/_5), (5, 3^4/_5), (6, 4^2/_5), (7, 5)$

Notice that as the $x$ coordinates go up by one, the $y$ coordinates simply go up by the slope of the line

This is the key insight in the DDA algorithm

# The DDA Algorithm (cont…)

When the slope of the line is between -1 and 1 begin at the first point in the line and, by incrementing the $x$ coordinate by 1, calculate the corresponding $y$ coordinates as follows:

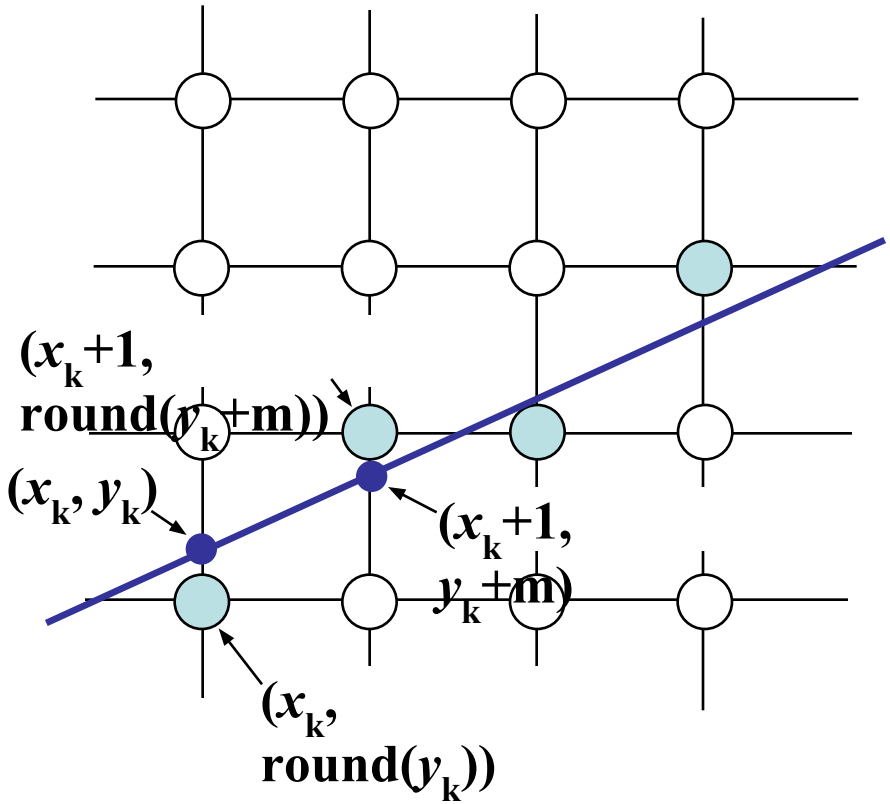
$$y_{k+1} = y_k + m$$

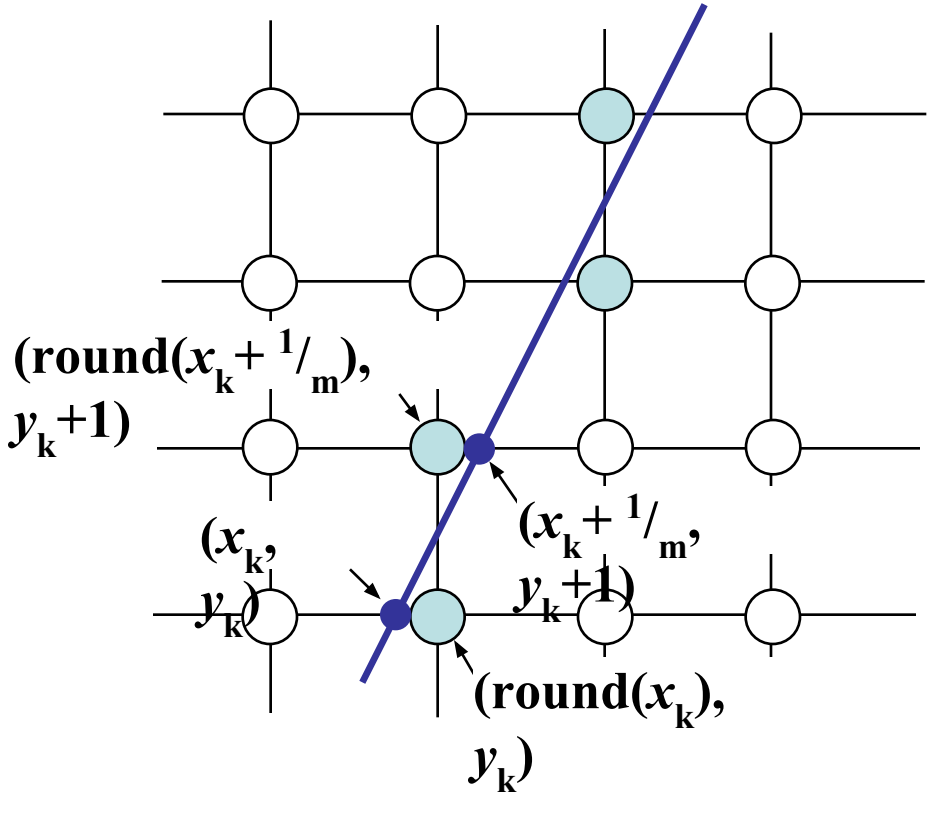
When the slope is outside these limits, increment the $y$ coordinate by 1 and calculate the corresponding $x$ coordinates as follows:

$$x_{k+1} = x_k + \frac{1}{m}$$

# The DDA Algorithm (cont…)

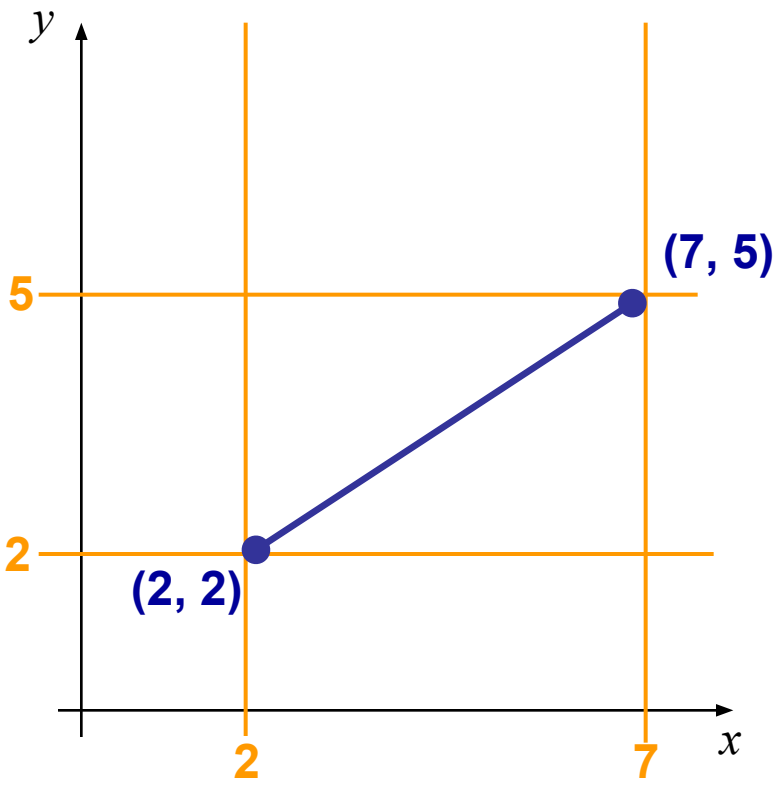
Again the values calculated by the equations used by the DDA algorithm must be rounded to match pixel values

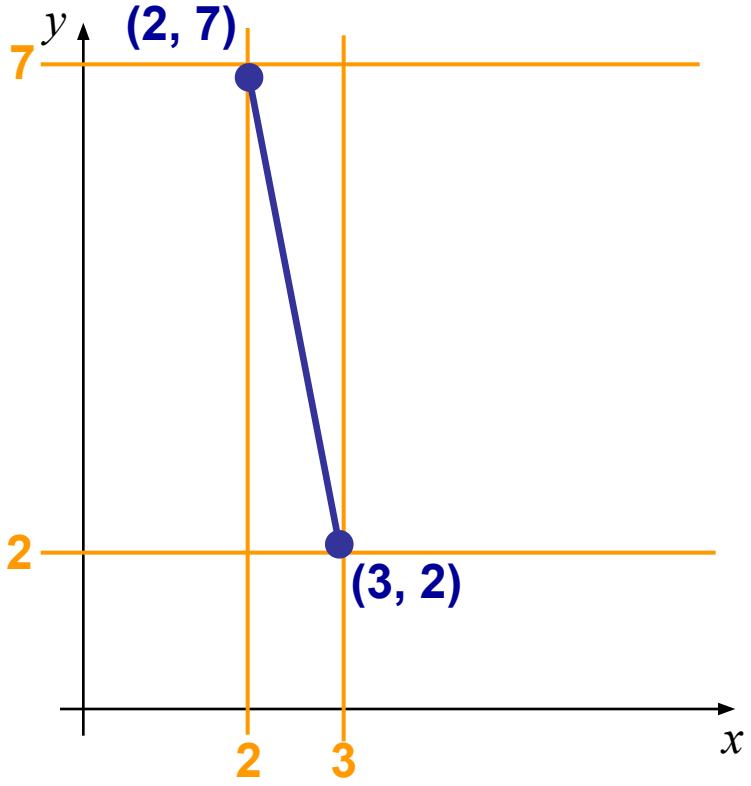


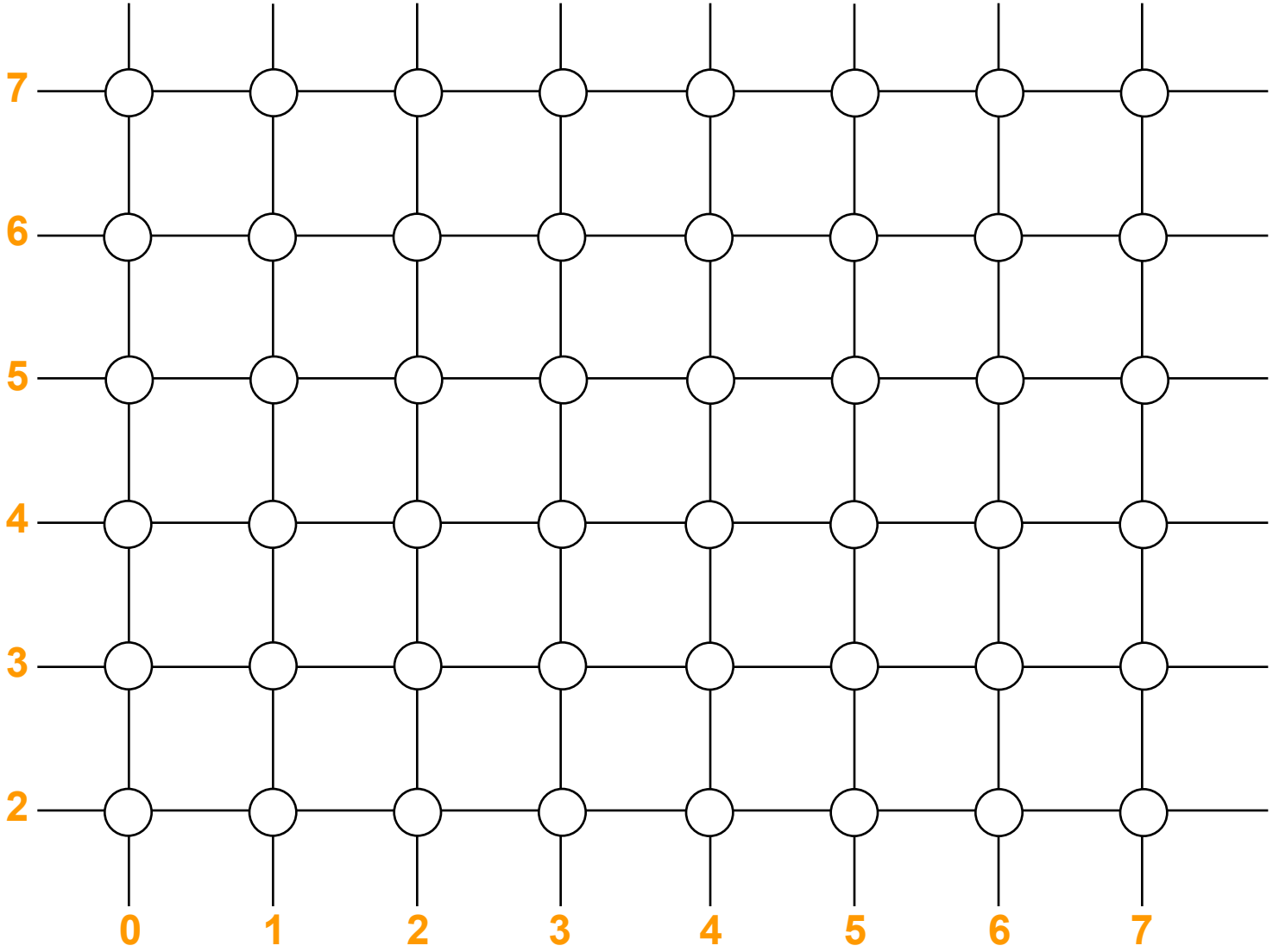
# DDA Algorithm Example

Let's try out the following examples:



# DDA Algorithm Example (cont…)



# The DDA Algorithm Summary

The DDA algorithm is much faster than our previous attempt

- In particular, there are no longer any multiplications involved

However, there are still two big issues:

- Accumulation of round-off errors can make the pixelated line drift away from what was intended
- The rounding operations and floating point arithmetic involved are time consuming